A REAL TIME DRIVER DROWSINESS DETECTION AND ALERT SYSTEM

A project report submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (AI & ML)

by

V.MOUNIKA JAYA	22BFA33138
K.CHERISHMA	22BFA33097
JADALA VAISHNAVI	22BFA33094
MUDDUKUPPAM DILLI PRASAD	22BFA33123

Under the guidance of

Dr. J. Sarada

Associate Professor

Department of CSE (AI & ML)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

SRI VENKATESWARA COLLEGE OF ENGINEERING

(AUTONOMOUS)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAAC with 'A' grade)
Karakambadi Road, TIRUPATI – 517507

2022 - 2026

SRI VENKATESWARA COLLEGE OF ENGINEERING

(AUTONOMOUS)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAAC with 'A' Grade)
Karakambadi Road, TIRUPATI – 517507.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

CERTIFICATE

This is to certify that the project work entitled, **"A REAL TIME DRIVER DROWSINESS DETECTION AND ALERT SYSTEM"** is a bonafide record of the project work done and submitted by

V.MOUNIKA JAYA	**22BFA33138**
K.CHERISHMA	**22BFA33097**
JADALA VAISHNAVI	**22BFA33094**
MUDDUKUPPAM DILLI PRASAD	**22BFA33123**

under my guidance and supervision for the partial fulfillment of the requirements for the award of B.Tech degree in **COMPUTER SCIENCE AND ENGINEERING (AI & ML).** This project is the result of our own effort and that it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above

GUIDE **HEAD OF THE DEPARTMENT**

External Viva-Voce Exam held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

We hereby declare that the project entitled **"A REAL TIME DRIVER DROWSINESS DETECTION AND ALERT SYSTEM",** done by us under the esteemed guidance of **Dr. J. Sarada, Associate Professor** and is submitted in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Technology in Computer Science and Engineering (AI & ML).**

This report is the result of our own effort and it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above.

Date:

Place:

V.MOUNIKA JAYA	**22BFA33138**
K.CHERISHMA	**22BFA33097**
JADALA VAISHNAVI	**22BFA33094**
MUDDUKUPPAM DILLI PRASAD	**22BFA33123**

ACKNOWLEDGEMENT

We are thankful to our guide **Dr. J. Sarada, Associate Professor, Department of CSE (AI & ML)** for her valuable guidance and encouragement. Her helping attitude and suggestions have helped in the successful completion of the project report.

We would like to express our gratefulness and sincere thanks to **Dr. R. Swathi, Professor & HOD, Department of CSE (AI & ML),** for her kind help and encouragement during the course of study and in the successful completion of the project work.

We have great pleasure in expressing out hearty thanks to our beloved Principal **Dr. Vijaya Gunturu** for spending his valuable time with us to complete this project.

Successful completion of any project cannot be done without proper support and encouragement. We sincerely thank the management for providing all the necessary facilities during course of study.

We would like to thank our parents and friends, who have the greatest contributions in all our achievements, for the great care and blessings in making us successful in all our endeavors.

V.MOUNIKA JAYA 22BFA33138
K.CHERISHMA 22BFA33097
JADALA VAISHNAVI 22BFA33094
MUDDUKUPPAM DILLI PRASAD 22BFA33123

ABSTRACT

To enhance road safety, this project proposes a real-time drowsiness detection system using Deep Learning with Convolutional Neural Networks (CNNs). The system analyzes a drivers facial expressions and eye movements to determine signs of fatigue and alertness. The model is trained on a dataset of facial images categorized into drowsy and non-drowsy states. Using CNN, the system extracts key facial features such as eye closure, yawning, and head position to detect drowsiness. A camera continuously captures frames of the driver's face, and the trained CNN model classifies them in real-time. If drowsiness is detected, an alert mechanism (such as a voice alarm) is triggered to wake the driver. The system is implemented using OpenCV, TensorFlow/Keras, and Python. The proposed solution is highly effective in reducing accidents caused by driver fatigue, providing a low-cost and scalable approach to road safety.

CONTENTS

Abstract	**i**
Table of Contents	**ii**
List of figures	**iv**
List of Tables	**v**
Abbreviations	**Vi**

Chapter No.	Description	Page No.
1	**INTRODUCTION**	**1-2**
2	**PROJECT DESCRIPTION**	**3-5**
	2.1 Problem Definition	3
	2.2 Project Details	3-4
	2.3 Scope of the Project	4-5
3	**SYSTEM ANALYSIS**	**6-7**
	3.1 Existing System	6
	3.1.1 Disadvantages	6
	3.2 Proposed System	7
	3.2.1 Advantages	7
4	**ENVIRONMENTS**	**8-10**
	4.1 Software Requirements.	8
	4.2 Hardware Requirements	8
	4.3 Software Features and Dependencies	9-10

5	**SYSTEM DESIGN**	**11-28**
	5.1 System Architecture	11-14
	5.2 UML Diagrams	14-28
	5.2.1 Class Diagram	16-18
	5.2.2 Use Case Diagram	18-21
	5.2.3 Sequence Diagram	21-23
	5.2.4 Activity Diagram	23-25
	5.2.5 Component Diagram	26
	5.2.6 Deployment Diagram	27
	5.2.7 Collaboration Diagram	28
6	**SYSTEM IMPLEMENTATION**	**29-33**
	6.1 Implementation Process	29-31
	6.2 Module Explanation	31-33
7	**SYSTEM STUDY AND TESTING**	**34-39**
	7.1 Feasibility Study	34-35
	7.2 Software Testing	35-39
8	**SOURCE CODE**	**40-53**
9	**SCREEN LAYOUTS**	**54-56**
10	**FUTURE ENHANCEMENTS**	**57-58**
11	**CONCLUSION**	**59**
12	**BIBILOGRAPHY**	**60**

LIST OF FIGURES

Fig. No.	Figure Name	Page No.
5.1	System Architecture	14
5.2.1	Class diagram	18
5.2.2	Use Case diagram	21
5.2.3	Sequence diagram	23
5.2.4	Activity diagram	25
5.2.5	Component diagram	26
5.2.6	Deployment diagram	27
5.2.7	Collaboration Diagram	28
6.2.1	List of Modules	31
9.1	Interface Layout	54-55
9.1.1	Web Interface	54
9.1.2	Detection GUI Window	55
9.2	Result	56
9.2.1	Eye State Detection	56
9.2.2	Yawning Detection	56

LIST OF TABLES

Table No.	Table Name	Page No.
7.2.1	Unit testing	36
7.2.2	Integration testing	37
7.2.3	System testing	38
7.2.4	Acceptance testing	39

ABBREVIATIONS

AI	-	Artificial Intelligence
CNN	-	Convolutional Neural Networks
API	-	Application Programming Interface
GUI	-	Graphical User Interface
CPU	-	Central Processing Unit
CSS	-	Cascading Style Sheets
OS	-	Operating System
DL	-	Deep Learning
MOR	-	Mouth Opening Ratio
GPU	-	Graphics Processing Unit
SVGA	-	Super Video Graphics Array
HTML	-	HyperText Markup Language
EAR	-	Eye Aspect Ratio
OpenCV	-	Open-Source Computer Vision Library
NumPY	-	Numerical Python
TensorFlow	-	Machine Learning Framework by Google
DNN	-	Deep Neural Network

Chapter 1
INTRODUCTION

Driver drowsiness is one of the leading causes of road accidents across the world. Long driving hours, lack of sleep, monotonous road conditions, and fatigue can make drivers feel sleepy while driving. When a driver becomes drowsy, their ability to concentrate, react quickly, and make proper decisions decreases significantly. This increases the risk of accidents, which may result in serious injuries, loss of life, and property damage. Therefore, detecting driver drowsiness at an early stage and alerting the driver is very important for improving road safety.

Many studies and road safety reports indicate that a large percentage of road accidents occur due to driver fatigue. Drivers who feel sleepy may experience slower reaction times, blurred vision, and reduced attention to road conditions. In some cases, drivers may even fall asleep for a few seconds, which is known as a microsleep. Even a few seconds of inattention while driving at high speed can lead to dangerous situations. Because of these risks, researchers and engineers are working on intelligent systems that can monitor the driver's condition and provide warnings before accidents occur.

Traditional driver monitoring systems mainly rely on physiological signals such as Electroencephalography (EEG), Electrocardiography (ECG), and other biometric sensors. These systems measure brain signals, heart rate, and other biological parameters to detect signs of fatigue. Although these methods can provide accurate results, they require special sensors to be attached to the driver’s body. This makes them uncomfortable for long-term use and also increases the cost of the system. As a result, such systems are not widely used in everyday vehicles.

To overcome these limitations, modern driver monitoring systems use computer vision and artificial intelligence techniques. These systems rely on cameras and image processing algorithms to analyze the driver’s facial features and behavior. Important indicators of drowsiness include eye closure, blinking

rate, yawning frequency, and head movements. By analyzing these visual features, it is possible to detect whether a driver is alert or becoming drowsy. Vision-based systems are non-intrusive, cost-effective, and easier to implement compared to physiological monitoring systems.

Deep learning techniques, particularly Convolutional Neural Networks (CNNs), have shown great success in image recognition and facial feature detection tasks. CNN models are capable of automatically extracting important features from images and learning patterns that indicate different states of alertness. When trained with a large dataset of driver images, CNN models can accurately distinguish between alert and drowsy states. These models can also process images in real time, making them suitable for driver monitoring applications.

In this project, a real-time driver drowsiness detection and alert system is developed using deep learning and computer vision techniques. The system uses a camera installed in front of the driver to continuously capture video frames. These frames are processed using image preprocessing techniques, and the relevant facial features are extracted using a CNN-based model. The system then classifies the driver's condition as either alert or drowsy based on the extracted features.

If the system detects signs of drowsiness, it immediately triggers an alert such as an audio alarm to warn the driver. This helps the driver regain attention and take necessary actions, such as stopping the vehicle or taking a rest. By providing real-time monitoring and alerts, the proposed system helps reduce the chances of accidents caused by driver fatigue.

The implementation of such intelligent monitoring systems can play an important role in improving road safety and reducing accident rates. With the increasing development of artificial intelligence, these systems can be integrated into modern vehicles and smart transportation systems. This project demonstrates how deep learning and computer vision technologies can be used effectively to create a practical and efficient solution for detecting driver drowsiness and enhancing road safety.

Chapter 2

PROJECT DESCRIPTION

2.1 PROBLEM DEFINITION:

Driver drowsiness is a major factor contributing to road accidents worldwide. When drivers become tired or sleepy, their ability to concentrate, react quickly, and make safe driving decisions decreases significantly. This condition can lead to delayed reactions, lane drifting, and even falling asleep while driving, which increases the risk of serious accidents. Many accidents occur because drivers fail to recognize their own level of fatigue or are unable to take timely action.

Existing driver monitoring systems often rely on physiological sensors such as Electroencephalography (EEG) or Electrocardiography (ECG) to detect fatigue. Although these systems can provide accurate measurements, they require specialized equipment attached to the driver's body, making them uncomfortable and expensive for regular use. Additionally, some vision-based systems face challenges such as poor lighting conditions, head movements, and variations in facial orientation, which can reduce detection accuracy.

Therefore, there is a need for an efficient, reliable, and real-time driver monitoring system that can detect signs of drowsiness without using intrusive sensors. The system should continuously monitor the driver's facial features, accurately identify signs of fatigue such as eye closure or yawning, and provide immediate alerts to prevent accidents. Developing such a system using modern technologies like computer vision and deep learning can help improve road safety and reduce accidents caused by driver fatigue.

2.2 PROJECT DETAILS

The Real-Time Driver Drowsiness Detection and Alert System is designed to monitor the driver's alertness level while driving and provide warnings when signs of fatigue are detected. Driver drowsiness is one of the major causes of road accidents, especially during long-distance driving or night-time travel. This project aims to reduce such accidents by continuously analyzing the driver's facial features and detecting early signs of sleepiness.

The system uses a camera placed in front of the driver to capture real-time video frames. These frames are processed using image processing and deep learning techniques to identify important facial features such as eye closure, blinking rate, yawning, and head movement. These features are strong indicators of driver fatigue. The captured images are preprocessed and then passed to a trained Convolutional Neural Network (CNN) model that classifies the driver's state as either alert or drowsy.

When the model detects that the driver is becoming drowsy, the system immediately triggers an audio alert to wake the driver and help them regain attention. This real-time alert mechanism helps in preventing potential accidents by warning the driver at the right time.

The system is implemented using Python programming language with the help of libraries such as OpenCV for image processing, TensorFlow/Keras for deep learning, and Streamlit for the user interface. The trained CNN model processes each frame captured by the camera and makes predictions in real time.

Overall, the proposed system provides a cost-effective, non-intrusive, and efficient solution for monitoring driver behavior. It can be integrated into vehicles to enhance road safety and reduce accidents caused by driver fatigue.

2.3 SCOPE OF THE PROJECT

The scope of the Real-Time Driver Drowsiness Detection and Alert System is to develop an intelligent system that can monitor the driver's alertness level and detect signs of fatigue in real time. The project focuses on using computer vision and deep learning techniques to analyze facial features such as eye closure, yawning, and head movement to determine whether the driver is alert or drowsy.

The system captures live video using a camera placed in front of the driver and processes the video frames using image processing techniques. A trained Convolutional Neural Network (CNN) model is used to classify the driver's state. When the system detects signs of drowsiness, it immediately triggers an audio alert to warn the driver and help prevent accidents.

This project mainly focuses on developing a low-cost, non-intrusive, and real-time monitoring system that can be easily implemented in vehicles. The system is

designed to work using commonly available hardware such as webcams and standard computing devices. It aims to improve road safety by providing early warnings to drivers when they begin to feel sleepy.

In the future, the system can be expanded by integrating it with smart vehicles, mobile applications, and IoT-based transportation systems. Additional features such as GPS tracking, automatic vehicle control, and cloud-based monitoring can also be added to enhance the effectiveness of the system.

Chapter 3
SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Existing driver drowsiness detection systems mainly focus on monitoring the physical or physiological condition of the driver to determine fatigue. Many traditional systems use physiological sensors such as Electroencephalography (EEG), Electrocardiography (ECG), and heart rate monitoring devices to measure brain activity, heart rhythm, and other biological signals. These methods can detect fatigue with reasonable accuracy, but they require sensors to be attached to the driver's body, making them uncomfortable and difficult to use during long journeys.

Some existing systems also use camera-based monitoring techniques to observe the driver's facial features such as eye blinking, eye closure, and mouth movement. These systems apply image processing and machine learning algorithms to identify signs of drowsiness. However, many of these systems face challenges such as poor lighting conditions, variations in facial orientation, and difficulties in accurately detecting facial features when the driver moves their head.

In addition, some traditional machine learning models like Support Vector Machines (SVM) are used for classification. These models require manual feature extraction and may not perform well when the dataset contains noise or overlapping features.

3.1.1 Disadvantages

- Requires expensive sensors and specialized equipment.
- Sensors attached to the driver's body can be uncomfortable.
- Performance may decrease under poor lighting conditions.
- Traditional algorithms may not handle large and complex datasets effectively.
- Limited accuracy in detecting real-time driver fatigue.

3.2 PROPOSED SYSTEM

The proposed system introduces a real-time driver drowsiness detection and alert system using deep learning techniques. The system is designed to continuously monitor the driver's facial expressions and detect signs of fatigue while driving. It uses a camera placed in front of the driver to capture live video frames of the driver's face during driving.

The captured video frames are processed using image preprocessing techniques such as resizing, normalization, and feature enhancement to prepare the images for analysis. A trained Convolutional Neural Network (CNN) model is then used to extract important facial features such as eye closure, yawning, blinking rate, and head movement, which are common indicators of driver drowsiness.

The CNN model analyzes these features and classifies the driver’s state into two categories: alert or drowsy. If the system detects that the driver is becoming drowsy, it immediately activates an alert mechanism, such as an audio alarm, to warn the driver and help them regain attention.

The proposed system is implemented using Python, OpenCV, TensorFlow/Keras, and Streamlit. It provides a non-intrusive, cost-effective, and efficient solution for monitoring driver behavior in real time. Compared to traditional systems, this approach improves accuracy and usability while reducing the need for expensive hardware or wearable sensors.

3.2.1 Advantages

- Provides **real-time monitoring** of the driver’s condition.
- Uses **deep learning techniques** for higher accuracy.
- Does not require sensors attached to the driver’s body.
- Provides **instant alerts** to prevent accidents caused by fatigue.
- Low-cost and easy to implement using camera-based systems.

Chapter 4

ENVIRONMENTS

4.1 SOFTWARE REQUIREMENTS

- Operating system : Windows 11
- Programming Language : Python
- Framework : Streamlit
- Development Environment : VSCode

4.2 HARDWARE REQUIREMENTS

- Processor : Intel i3 or above
- RAM : 4 GB or above
- Hard Disk : 256 GB
- Key Board : Standard Windows Keyboard
- Mouse : Two or Three Button Mouse
- Monitor : SVGA
- CPU Speed : 1.6 GHz or higher

4.3 SOFTWARE FEATURES AND DEPENDENCIES

Python is widely used for developing machine learning and deep learning applications due to its simplicity and availability of powerful libraries. In this project, a real-time driver drowsiness detection system is developed using deep learning techniques. Various Python libraries are used to perform tasks such as image processing, feature extraction, model training, and real-time alert generation. These tools make the system efficient, accurate, and easy to implement.

• TensorFlow

TensorFlow is an open-source deep learning framework developed by Google. It is used to build and train the Convolutional Neural Network (CNN) model that detects whether the driver is drowsy or alert.

Key features:

- Provides support for building deep learning models like CNN
- Supports both CPU and GPU for faster computation
- Offers high accuracy in image classification tasks
- Can handle large datasets efficiently
- Provides tools for model saving and deployment

• Keras

Keras is a high-level neural network API that runs on top of TensorFlow. It simplifies the process of designing and training deep learning models.

Key features:

- Easy to use and understand
- Helps in quick model development
- Supports multiple layers such as convolutional, pooling, and dense layers
- Reduces coding complexity
- Ideal for beginners and research projects

• OpenCV

OpenCV (Open Source Computer Vision Library) is used for real-time image and video processing. It plays a major role in detecting the driver's face and eyes.

Key features:

- Captures live video using a camera
- Detects face and eye regions
- Converts images into grayscale for better processing
- Helps in drawing bounding boxes around detected features
- Works efficiently in real-time applications

• NumPy

NumPy is a fundamental library used for numerical operations and array handling in Python.

Key features:

- Efficient handling of multi-dimensional arrays
- Supports mathematical and logical operations
- Used to process image pixel data
- Improves performance of computations

• Streamlit

Streamlit is used to create a simple and interactive user interface for the application.

Key features:

- Easy to build web-based interface
- Displays real-time results
- Allows user interaction
- Lightweight and fast

Chapter 5

SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

The system architecture of the Real-Time Driver Drowsiness Detection and Alert System describes how different components of the system work together to detect driver fatigue and generate alerts. The architecture is designed to continuously monitor the driver's face using a camera and analyze the captured images using deep learning techniques. The system processes each stage sequentially, starting from image capture to alert generation, ensuring real-time detection of drowsiness.

The architecture consists of several important modules such as Data Acquisition, Image Preprocessing, Feature Extraction, Classification, and Alert Generation. Each module performs a specific function that contributes to the overall performance of the system.

1. Data Acquisition Module:

The first stage of the system architecture is the Data Acquisition Module. In this stage, a camera is placed in front of the driver to capture real-time video of the driver's face while driving. The camera continuously records video frames at regular intervals and sends these frames to the processing unit.

The captured images provide important visual information about the driver's facial expressions, eye movements, and head position. These frames act as the input data for the system. In addition to live video capture, the system may also use a pre-collected dataset of labeled driver images during the training phase to build an accurate deep learning model.

2. Image Preprocessing Module:

After the video frames are captured, they are sent to the Image Preprocessing Module. Raw images may contain noise, variations in lighting, or unnecessary background information that can affect model performance. Therefore, preprocessing is required to improve image quality and prepare the images for analysis.

The preprocessing stage involves several operations such as:

- Resizing images to a fixed dimension required by the CNN model

- Normalizing pixel values to maintain consistent input scale
- Converting images to grayscale if necessary to reduce complexity
- Noise reduction and image enhancement

These preprocessing techniques help the system focus only on the important facial features and improve the accuracy of the model.

3. Feature Extraction Module:

The Feature Extraction Module is responsible for identifying key facial characteristics that indicate driver fatigue. Instead of manually extracting features, the system uses a Convolutional Neural Network (CNN) to automatically detect relevant patterns from the input images.

The CNN model analyzes the driver's face and extracts features such as:

- Eye closure or blinking patterns
- Yawning detection
- Head tilting or nodding
- Facial expression changes

The convolutional layers of the CNN detect edges, shapes, and patterns within the image, while deeper layers learn complex relationships between different facial features. This automated feature extraction improves detection accuracy and reduces the need for manual intervention.

4. Classification Module:

Once the features are extracted, they are passed to the Classification Module. In this stage, the trained CNN model evaluates the extracted features and determines the driver's condition. The model classifies each frame into two categories:

Alert State – The driver is attentive and actively driving.

Drowsy State – The driver shows signs of fatigue such as closed eyes or yawning.

The classification process occurs continuously for every frame captured by the camera, allowing the system to monitor the driver in real time. If multiple frames indicate signs of drowsiness, the system confirms that the driver is in a fatigued state.

5. Alert Generation Module:

The final stage of the architecture is the Alert Generation Module. If the classification module detects that the driver is drowsy, the system immediately activates an alert mechanism. The alert is designed to wake the driver and regain their attention.

The alert system may include:

- Audio alarm to warn the driver
- Visual notification on the screen
- Message or warning signal

This immediate response helps the driver become aware of their fatigue and take corrective actions such as stopping the vehicle or taking a rest.

Overall Workflow

The complete workflow of the system architecture can be summarized as follows:

- Camera captures real-time driver images.
- Images are preprocessed to improve quality and consistency.
- CNN extracts important facial features from the images.
- The trained model classifies the driver as alert or drowsy.
- If drowsiness is detected, the system triggers an alert.

By integrating these modules, the proposed architecture provides an efficient, accurate, and real-time driver monitoring system that helps reduce road accidents caused by driver fatigue. The system is designed to be cost-effective and easily implementable in modern vehicles to enhance road safety.

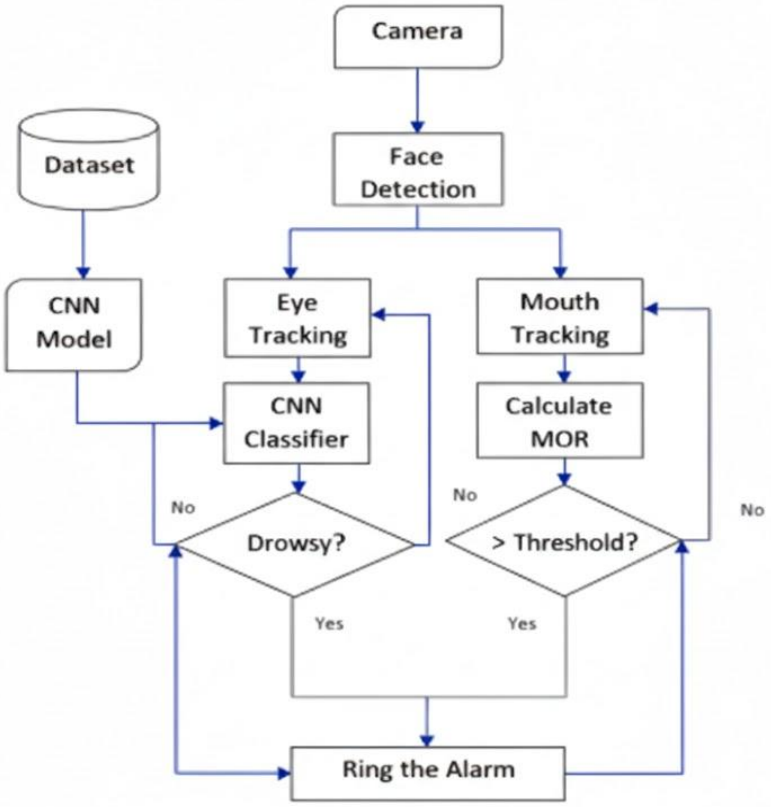


Fig 5.1: System architecture

5.2 UML DIAGRAMS

What is UML?

The Unified Modelling Language (UML) is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modelling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modelling of large and complex systems. The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects. Using the UML helps project teams

communicate, explore potential designs, and validate the architectural design of the software.

As the strategic value of software increases for many companies, the industry looks for techniques to automate the production of software and to improve quality and reduce cost and time-to-market. These techniques include component technology, visual programming, patterns and frameworks. Businesses also seek techniques to manage the complexity of systems as they increase in scope and scale. In particular, they recognize the need to solve recurring architectural problems, such as physical distribution, concurrency, replication, security, load balancing and fault tolerance. Additionally, the development for the World Wide Web, while making some things simpler, has exacerbated these architectural problems. The Unified Modelling Language (UML) was designed to respond to these needs. Simply, systems design refers to the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements, which can be done easily through UML diagrams.

Contents of UML:

In general, a UML diagram consists of the following features:

- ***Entities***

  These may be classes, objects, users, or systems behaviors.Relationship Lines that model the relationships between entities in the system.
- ***Generalization***

  A solid line with an arrow that points to a higher abstraction of the present item.
- ***Association***

  A solid line that represents that one entity uses another entity as part of its behaviour.
- ***Dependency***

A dotted line with an arrowhead that shows one entity depends on the behaviour of another entity.

Advantages:

- Provides standard for software development.
- Development time is reduced.
- The past faced issues by the developers are no longer exists.

- Has large visual elements to construct and easy to follow.
- To establish an explicit coupling between concepts and executable code.
- To creating a modeling language usable by both humans and machines UML defines several models for representing systems.

In this project six basic UML diagrams have been explained:

1. Class Diagram
2. Use Case Diagram
3. Sequence Diagram
4. Activity Diagram
5. Component Diagram
6. Deployment Diagram

5.2.1 Class Diagram

UML class diagrams model static class relationships that represent the fundamental architecture of the system. Note that these diagrams describe the relationships between classes, not those between specific objects instantiated from those classes. Thus, the diagram applies to all the objects in the system. These diagrams help in understanding the system's design at a high level and serve as a blueprint for implementation. They identify the system's major components, their attributes, operations, and the relationships between them. Class diagrams support object-oriented design principles and are essential during both analysis and design phases. They assist in detecting potential issues early in the development cycle by visualizing the structure. UML class diagrams are also useful for documentation and communication among team members and stakeholders

A class diagram consists of the following features:

- ***Classes***

These titled boxes represent the classes in the system and contain information about the name of the class, fields, methods and access specifies. Abstract roles of the Class in the system can also be indicated.

Interfaces

These titled boxes represent interfaces in the system and contain information about the name of the interface and its methods. Relationship Lines that model the relationships between classes and interfaces in the system.

- ***Dependency***

A dotted line with an open arrowhead that shows one entity depends on the behavior of another entity. Typical usages are to represent that one class instantiates another or that it uses the other as an input parameter

- ***Aggregation***

Represented by an association line with a hollow diamond at the tail end. An aggregation models the notion that one object uses another object without "owning" it and thus is not responsible for its creation or destruction.

- ***Inheritance***

A solid line with a solid arrowhead that points from a sub-class to a super class or from a sub-interface to its super-interface.

- ***Implementation***

A dotted line with a solid arrowhead that points from a class to the interface that it implements.

- ***Composition***

Represented by an association line with a solid diamond at the tail end. A composition models the notion of one object "owning" another and thus being responsible for the creation and destruction of another object.

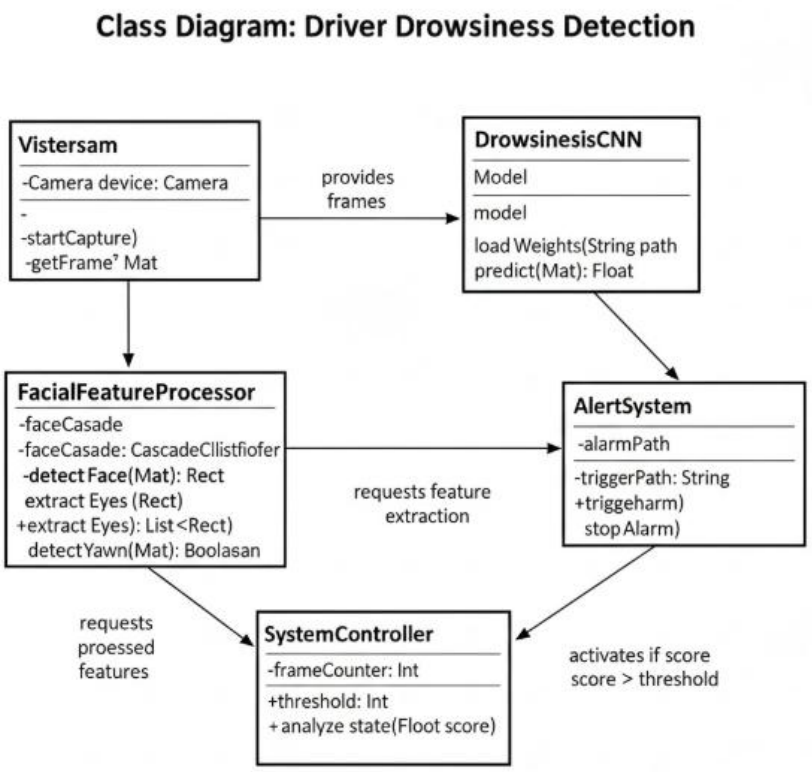


Fig 5.2.1: Class diagram

5.2.2 Use Case Diagram

A use case diagram in the Unified Modelling Language (UML) is a type of behavioural diagram defined by and created from a use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms.

A use case is a methodology used in system analysis to identify, clarify, and organize system requirements. The use case is made up of a set of possible sequences of interactions between systems and users in a particular environment and related to a particular goal. It consists of a group of elements (for example, classes and interfaces) that can be used together in a way that will have an effect larger than the sum of the separate elements combined. The use case should contain all system activities that have significance to the users. A use case can be thought of as a collection of possible scenarios related to a particular goal, indeed, the use case and goal are sometimes considered to be synonymous.

The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

Parts of use cases:

A use case describes a sequence of actions that provide something of measurable value to an actor and is drawn as a horizontal ellipse.

- ***Actors***

An actor is a person, organization, or external system that plays a role in one or more interactions with the system.

- ***System boundary boxes (optional)***

A rectangle is drawn around the use cases, called the system boundary box, to indicate the scope of the system. Anything within the box represents functionality that is in scope, and anything outside the box is not.

- ***Include***

In one form of interaction, a given use case may include another. "Include" is a directed relationship between two use cases, implying that the behavior of the included use case is inserted into the behavior of the including use case. The first use case often depends on the outcome of the included use case. This is useful for extracting truly common behaviours from multiple use cases into a single description. The notation is a dashed arrow from the including to the included use case, with the label "«include»". This usage resembles a macro expansion where the included use case behavior is placed inline in the base use case behavior. There are no parameters or return values. To specify the location in a flow of events in which the base use case includes the behavior of another, you simply write include followed by the name of the use case you want to include, as in the following flow for track order.

- ***Extend***

In another form of interaction, a given use case (the extension) may extend another. This relationship indicates that the behavior of the extension use case may be inserted in the extended use case under some conditions. The notation is a dashed arrow from the extension to the extended use case, with the label "«extend»". Notes or constraints may be associated with this relationship to illustrate the conditions under which this behavior will be executed. Modelers use the «extend» relationship to indicate use cases that are optional to the base use case. Depending on the modeler’s approach, "optional" may mean "potentially not executed with the base use case" or it may mean "not required to achieve the base use case goal".

- ***Generalization***

In the third form of relationship among use cases, a generalization/specialization relationship exists. A given use case may have common behaviours, requirements, constraints, and assumptions with a more general use case. In this case, describe them once, and deal with it in the same way, describing any differences in the specialized cases. The notation is a solid line ending in a hollow triangle drawn from the specialized to the more general use case (following the standard generalization notation).

- ***Associations***

Associations between actors and use cases are indicated in use case diagrams by solid lines. An association exists whenever an actor is involved with an interaction described by a use case. Associations are modelled as lines connecting use cases and actors to one another, with an optional arrowhead on one end of the line. The arrowhead is often used to indicate the direction of the initial invocation of the relationship or to indicate the primary actor within the use case. The arrowheads imply control flow and should not be confused with data flow.

Steps to draw use cases:

• Identifying actor

• Identifying use cases

• Review your use case for completeness

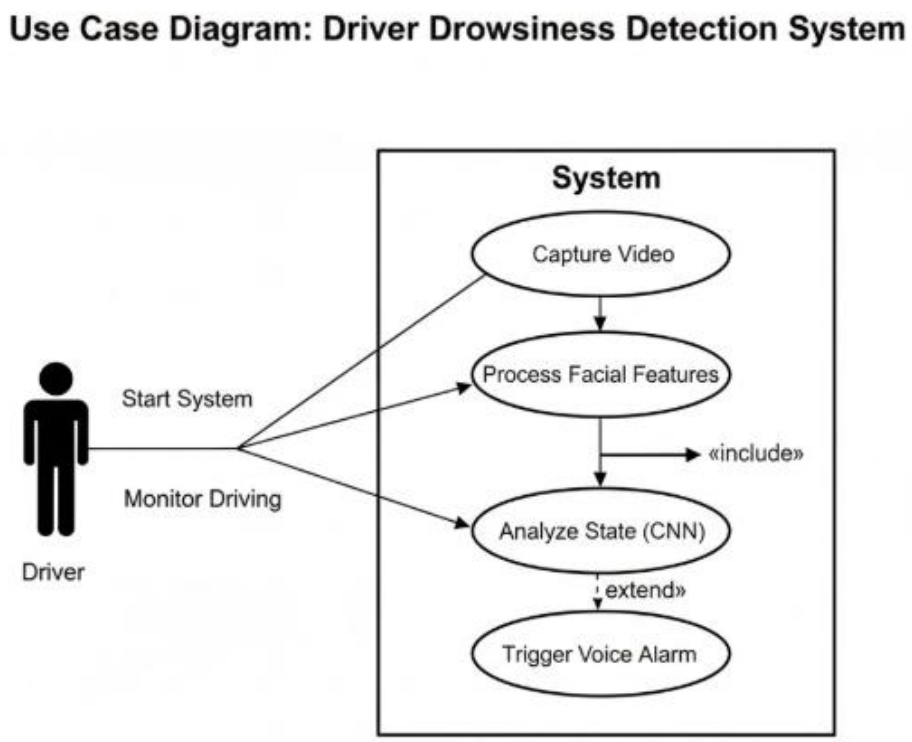


Fig 5.2.2: Use Case diagram

5.2.3 Sequence Diagram

A sequence diagram in Unified Modelling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram depicts the sequence of actions that occur in a system. The invocation of methods in each object, and the order in which the invocation occurs is captured in a sequence diagram.

This makes the sequence diagram a very useful tool to easily represent the dynamic behaviour of a system.

- ***Elements of sequence diagram***

The sequence diagram is an element that is used primarily to showcase the interaction that occurs between multiple objects. This interaction will be shown over a certain period of time. Because of this, the first symbol that is used is one that symbolizes the object.

- ***Lifeline***

A lifeline will generally be generated, and it is a dashed line that sits vertically, and the top will be in the form of a rectangle. This rectangle is used to indicate both the instance and the class. If the lifeline must be used to denote an object, it will be underlined.

- ***Messages***

To showcase an interaction, messages will be used. These messages will come in the form of horizontal arrows, and the messages should be written on top of the arrows. If the arrow has a full head, and it's solid, it will be called a synchronous call. If the solid arrow has a stick head, it will be an asynchronous call. Stick heads with dash arrows are used to represent return messages.

- ***Objects***

Objects will also be given the ability to call methods upon themselves, and they can add net activation boxes. Because of this, they can communicate with others to show multiple levels of processing. Whenever an object is eradicated or erased from memory, the "X" will be drawn at the lifeline's top, and the dash line will not be drawn beneath it. This will often occur as a result of a message. If a message is sent from the outside of the diagram, it can be used to define a message that comes from a circle that is filled in. Within a UML-based model, a Super step is a collection of steps which result from outside stimuli.

Steps to create a sequence diagram:

- Set the context for the interaction, whether it is a system, subsystem, operation or class.
- Set the stage for the interaction by identifying which objects play a role in the interaction.
- Set the lifetime for each object.
- Start with the message that initiates the interaction.
- Visualize the nesting of messages or the points in time during actual computation.
- Specify time and space constraints, adorn each message with timing marks and attach suitable time or space constraints.
- Specify the flow of control more formally, attach pre and post conditions to each message.

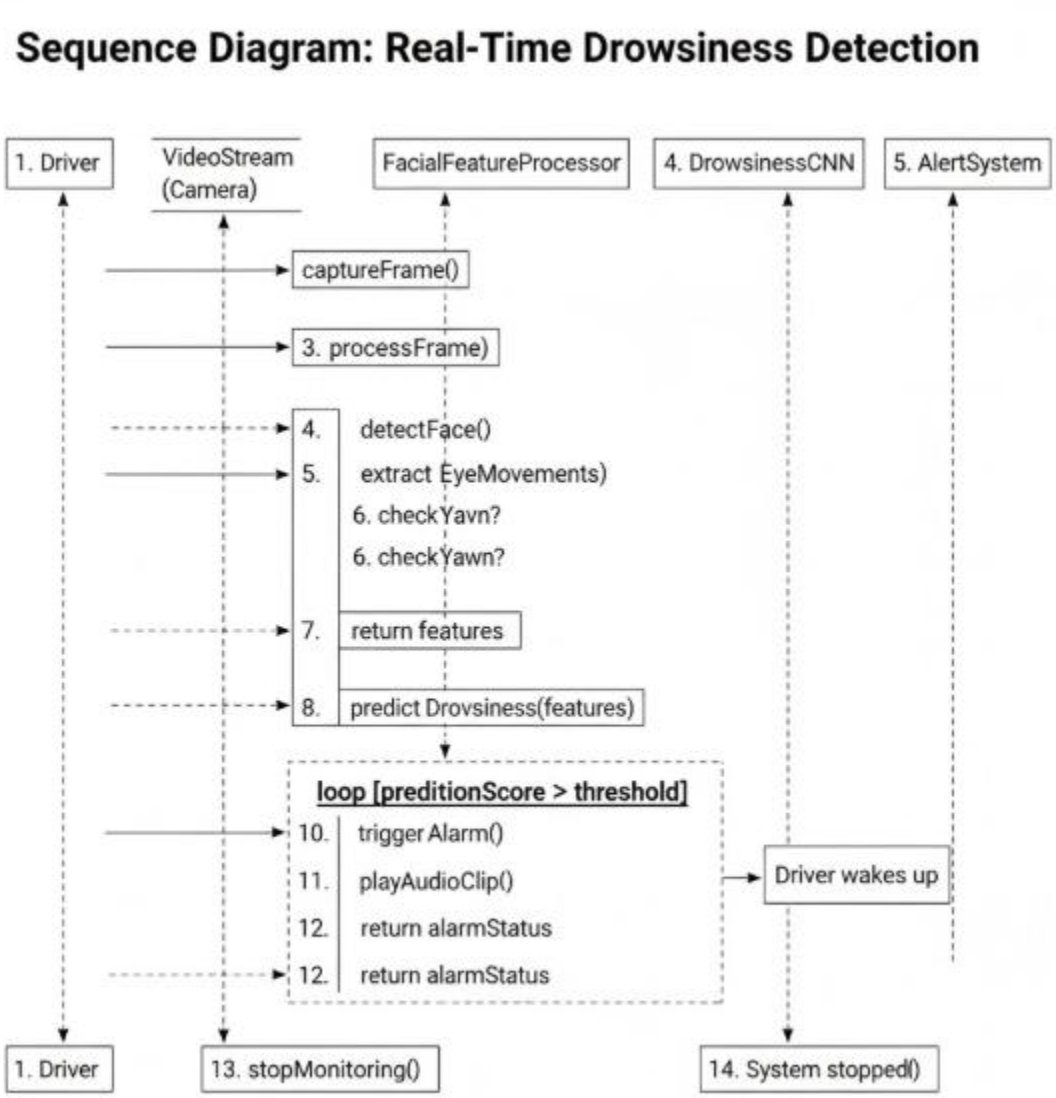


Fig 5.2.3: Sequence diagram

5.2.4 Activity Diagram

Activity diagram is another important diagram in UML to describe dynamic aspects of the system. Activity diagram is basically a flow chart to represent the flow from one activity to another activity. The activity can be described as an operation of the system. The control flow is drawn from one operation to another. This flow can be sequential,

branched or concurrent. Activity diagrams deal with all types of flow control by using different elements like fork, join etc.

How to draw activity diagram?

Activity diagrams are mainly used as a flow chart consisting of activities performed by the system. But activity diagrams are not exactly flow charts as they have some additional capabilities. These additional capabilities include branching, parallel flow, swim lanes etc. Before drawing an activity diagram, we must have a clear understanding about the elements used in activity diagrams. The main element of an activity diagram is the activity itself. An activity is a function performed by the system. After identifying the activities, we need to understand how they are associated with constraints and conditions. So before drawing an activity diagram, we should identify the following elements:

- Activities
- Association
- Conditions
- Constraints

The following are the basic notational elements that can be used to make up a diagram:

- ***Initial state***

An initial state represents a default vertex that is the source for a single transition to the default state of a composite state. There can be at most one initial vertex in a region. The outgoing transition from the initial vertex may have a behavior, but not a trigger or guard. It is represented by a filled circle, pointing to the initial state.

- ***Final state***

A special kind of state signifying that the enclosing region is completed. If the enclosing region is directly contained in a state machine and all other regions in the state machine are also completed, then it means that the entire state machine is completed. It is represented by a hollow circle containing a smaller filled circle, indicating the final state.

- ***Rounded rectangle***

It denotes a state. The top of the rectangle contains the name of the state. It can contain a horizontal line in the middle, below which the activities that are done in that state are indicated.

Arrow

It denotes transition. The name of the event (if any) causing this transition labels the arrow body.

Steps to construct activity diagram

• Identify the preconditions of the workflow

• Collect the abstractions that are involved in the operations

• Beginning at the operation's initial state, specify the activities and actions

• Use branching to specify conditional paths and iteration.

• Use forking and joining to specify parallel flows of control

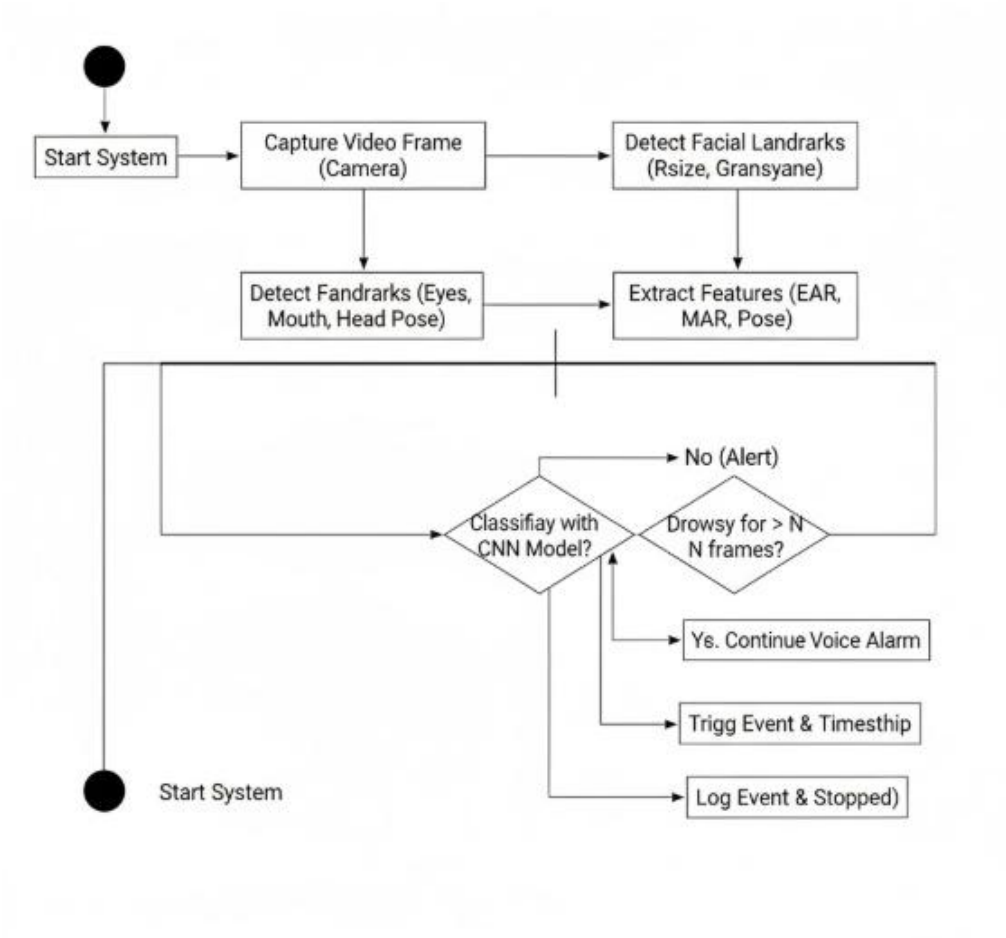


Fig 5.2.4: Activity diagram

5.2.5 Component Diagram

A component diagram is a type of UML (Unified Modeling Language) diagram that shows the structure of the components of a system and their interrelationships. It is a static view of the system that illustrates the physical and logical components of the system and how they work together. In a component diagram, components are represented as rectangles with the component name written inside. The interfaces of the component are shown as ports or small squares on the edges of the component. The relationships between the components are shown using connectors, such as associations or dependencies. Component diagrams are a powerful tool for system design, architecture, integration, and maintenance. They provide a clear and concise view of the system components and their relationships, which helps to ensure that the system is designed, built, and maintained correctly. Component diagrams are useful for visualizing the structure of a system and for identifying the relationships between the components.

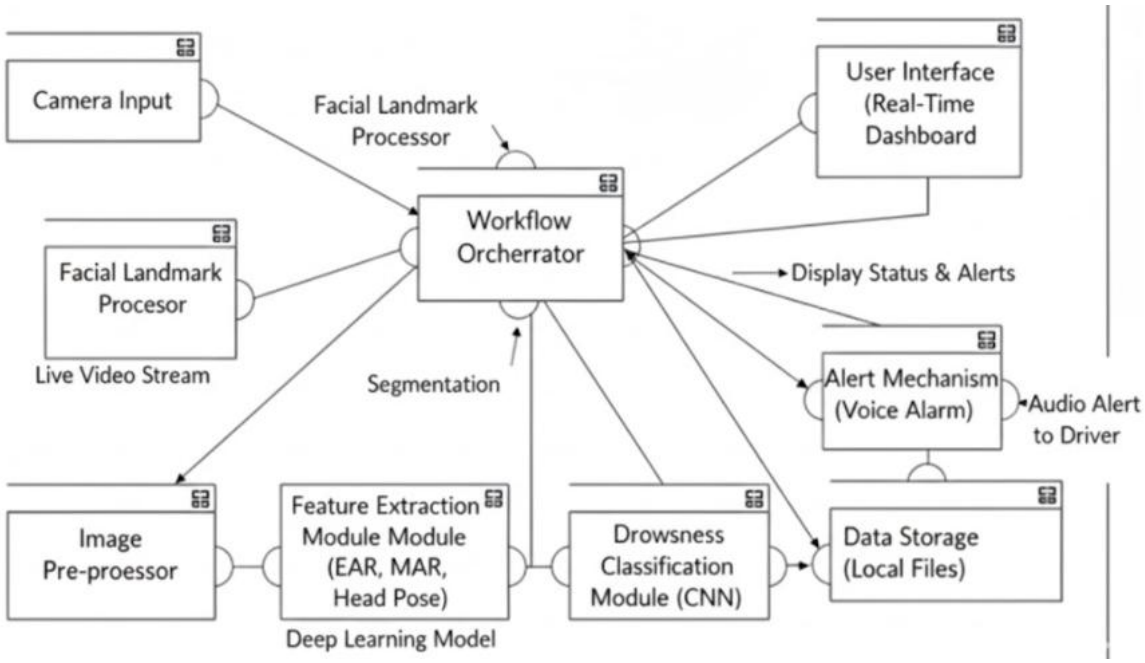


Fig 5.2.5: Component diagram

5.2.6 Deployment Diagram

The Deployment diagram represents the deployment view of a system. It is related to the component diagram. Because the components are deployed using the deployment diagrams. A deployment diagram consists of nodes. Nodes are nothing but physical hardware used to deploy the application. A deployment diagram is a type of UML (Unified Modeling Language) diagram that shows the physical deployment of software components and their relationships with hardware components in a system. It is a static view of the system that illustrates the physical architecture of the system and how the components are deployed on hardware devices or servers. In a deployment diagram, components are represented as nodes or boxes, and the relationships between the components are shown using connectors, such as associations or dependencies. Nodes represent physical devices, such as servers, and components represent software modules or applications. Deployment diagrams are useful for visualizing the physical deployment of software components in a system and for identifying potential issues or constraints in the deployment.

Deployment Diagram

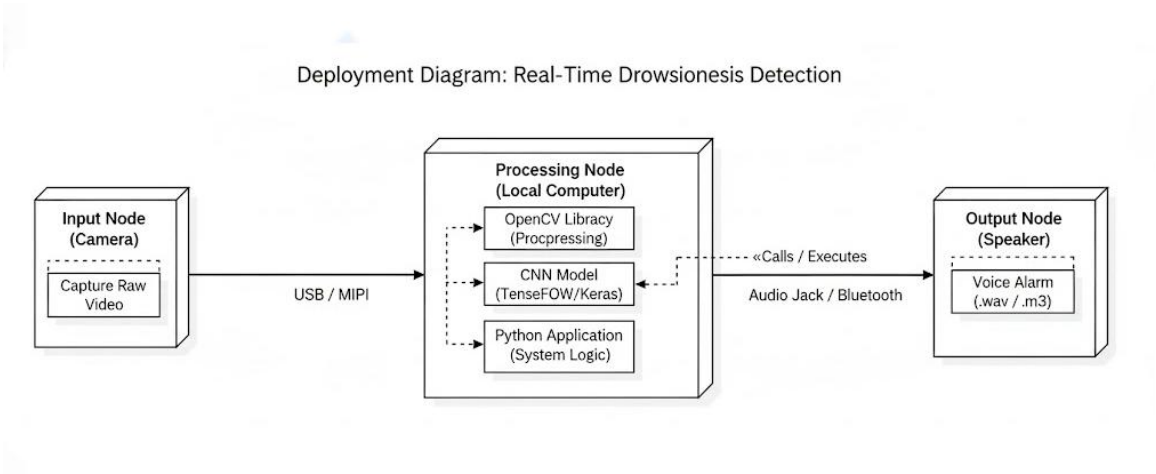


Fig 5.2.6: Deployment diagram

5.2.7 Collaboration Diagram

A collaboration diagram, also known as a communication diagram, is a type of UML diagram that illustrates the interactions between objects or components within a system, focusing on how they collaborate to perform a specific task. It emphasizes the structural organization of the system and the flow of messages exchanged between objects to achieve a particular functionality.

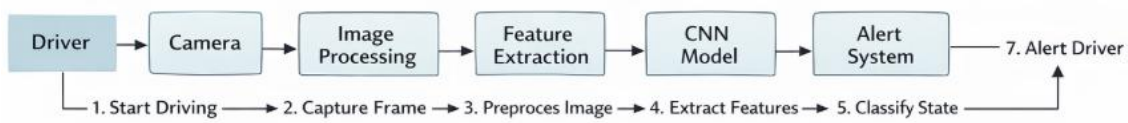


Fig 5.2.7: Collaboration Diagram

Chapter 6

SYSTEM IMPLEMENTATION

6.1 IMPLEMENTATION PROCESS

The implementation of the proposed Real-Time Driver Drowsiness Detection and Alert System using deep learning techniques is carried out through a structured sequence of phases. These phases include data collection, preprocessing, model development and training, evaluation, system integration, and deployment through a real-time monitoring interface. The goal of this implementation is to continuously monitor the driver's face and detect signs of fatigue such as eye closure, yawning, and head movement in order to trigger an alert and prevent accidents.

● **Data Collection and Preprocessing**

The dataset used in this project consists of a large number of driver facial images and video frames representing different driver states such as alert and drowsy. These images are collected from publicly available datasets and real-time recordings captured using cameras. The dataset includes variations in lighting conditions, head positions, facial expressions, and driver appearances to ensure that the model performs well under different real-world driving conditions.

Before training the model, the collected images undergo several preprocessing steps. All images are resized to a standard dimension suitable for the deep learning model input. Pixel values are normalized to a range between 0 and 1 to improve model performance. Data augmentation techniques such as rotation, horizontal flipping, brightness adjustment, and zooming are applied to increase dataset diversity and prevent overfitting. These preprocessing techniques help improve the model's ability to generalize and accurately detect drowsiness under different driving environments.

● **Model Selection – Convolutional Neural Network (CNN)**

A Convolutional Neural Network (CNN) is selected for this project because of its strong performance in image classification and facial feature detection tasks. CNN models are capable of automatically extracting important visual features from images such as edges, shapes, and patterns that are useful for detecting driver fatigue.

The CNN model is designed with multiple layers including convolutional layers, pooling layers, and fully connected layers. The convolutional layers extract facial features such as eye shape, mouth position, and head orientation. Pooling layers reduce the spatial dimensions of the feature maps while preserving important information. Finally, the fully connected layers perform classification to determine whether the driver is alert or drowsy.

● **Training the Model**

The CNN model is trained using labeled images representing both drowsy and non-drowsy driver states. The dataset is divided into training, validation, and testing sets (typically 80:10:10) to ensure proper evaluation of the model. The training process uses the categorical cross-entropy loss function and the Adam optimizer to update the model parameters and improve classification accuracy.

Several techniques are applied to enhance the training process and prevent overfitting, such as dropout layers, early stopping, and batch normalization. The model is trained for multiple epochs until the loss decreases and the accuracy stabilizes. GPU acceleration is used during training to speed up the computation process and handle large datasets efficiently.

● **Evaluation**

After the training process is completed, the model is evaluated using a separate test dataset that was not used during training. The evaluation process measures the performance of the model using metrics such as accuracy, precision, recall, F1-score, and confusion matrix analysis.

These evaluation metrics help determine how effectively the system can detect driver drowsiness. Training and validation accuracy and loss graphs are also analyzed to ensure that the model is properly trained and not overfitting. A high classification accuracy indicates that the model can reliably distinguish between alert and drowsy states.

● **System Integration**

Once the model is trained and validated, it is integrated into a real-time driver monitoring system. The system uses OpenCV to capture live video frames from a camera placed in front of the driver. Each captured frame is preprocessed and passed to the trained CNN model for prediction.

The system continuously analyzes the driver's facial features such as eye closure, blinking rate, yawning frequency, and head tilt. Based on the prediction results, the system determines whether the driver is alert or showing signs of fatigue. The processing is performed in real time to ensure quick detection and response.

● **Alert Generation and Deployment**

When the system detects that the driver is in a drowsy state, it immediately activates an alert mechanism such as an audio alarm or visual notification. The alert warns the driver and encourages them to regain attention or take a break from driving.

The entire system is implemented using Python, OpenCV, TensorFlow/Keras, and Streamlit for building the user interface. The interface displays the real-time video feed along with the driver status (Alert or Drowsy). The system can be deployed on a computer or embedded device installed in vehicles.

6.2 MODULE IMPLEMENTATION

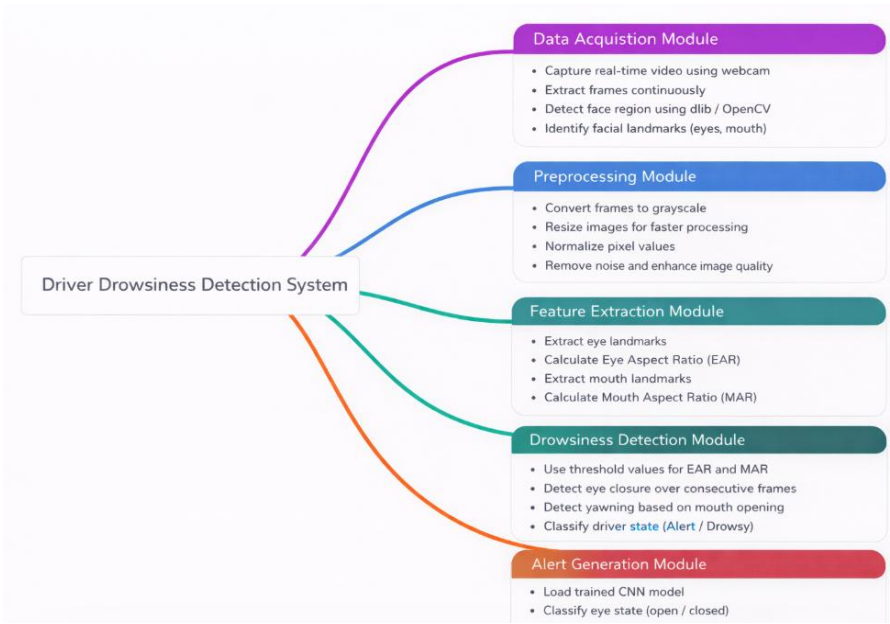


Fig 6.2.1 List of Modules

The Real-Time Driver Drowsiness Detection and Alert System describes how different components of the system work together to detect driver fatigue and generate alerts. The architecture is designed to continuously monitor the driver's face using a camera

and analyze the captured images using deep learning techniques. The system processes each stage sequentially, starting from image capture to alert generation, ensuring real-time detection of drowsiness.

The architecture consists of several important modules such as Data Acquisition, Image Preprocessing, Feature Extraction, Classification, and Alert Generation. Each module performs a specific function that contributes to the overall performance of the system.

1. Data Acquisition Module

The first stage of the system architecture is the Data Acquisition Module. In this stage, a camera is placed in front of the driver to capture real-time video of the driver's face while driving. The camera continuously records video frames at regular intervals and sends these frames to the processing unit.

The captured images provide important visual information about the driver's facial expressions, eye movements, and head position. These frames act as the input data for the system. In addition to live video capture, the system may also use a pre-collected dataset of labeled driver images during the training phase to build an accurate deep learning model.

2. Image Preprocessing Module

After the video frames are captured, they are sent to the Image Preprocessing Module. Raw images may contain noise, variations in lighting, or unnecessary background information that can affect model performance. Therefore, preprocessing is required to improve image quality and prepare the images for analysis.

The preprocessing stage involves several operations such as:

- Resizing images to a fixed dimension required by the CNN model
- Normalizing pixel values to maintain consistent input scale
- Converting images to grayscale if necessary to reduce complexity
- Noise reduction and image enhancement

These preprocessing techniques help the system focus only on the important facial features and improve the accuracy of the model.

3. Feature Extraction Module

The Feature Extraction Module is responsible for identifying key facial characteristics that indicate driver fatigue. Instead of manually extracting features, the

system uses a Convolutional Neural Network (CNN) to automatically detect relevant patterns from the input images.

The CNN model analyzes the driver's face and extracts features such as:

- Eye closure or blinking patterns
- Yawning detection
- Head tilting or nodding
- Facial expression changes

The convolutional layers of the CNN detect edges, shapes, and patterns within the image, while deeper layers learn complex relationships between different facial features. This automated feature extraction improves detection accuracy and reduces the need for manual intervention.

4. Classification Module

Once the features are extracted, they are passed to the Classification Module. In this stage, the trained CNN model evaluates the extracted features and determines the driver's condition. The model classifies each frame into two categories:

Alert State – The driver is attentive and actively driving.

Drowsy State – The driver shows signs of fatigue such as closed eyes or yawning.

The classification process occurs continuously for every frame captured by the camera, allowing the system to monitor the driver in real time. If multiple frames indicate signs of drowsiness, the system confirms that the driver is in a fatigued state.

5. Alert Generation Module

The final stage of the architecture is the Alert Generation Module. If the classification module detects that the driver is drowsy, the system immediately activates an alert mechanism. The alert is designed to wake the driver and regain their attention.

The alert system may include:

- Audio alarm to warn the driver
- Visual notification on the screen
- Message or warning signal

This immediate response helps the driver become aware of their fatigue and take corrective actions such as stopping the vehicle or taking a rest.

CHAPTER 7

SYSTEM STUDY AND TESTING

7.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase, and a business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis, the feasibility study of the proposed system is carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are:

1. Technical feasibility
2. Social feasibility
3. Economical feasibility

- **Technical feasibility**

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands being placed on the client. The developed system must have modest requirements, as only minimal or no changes are needed for implementing this system**.**

- **Social feasibility**

This aspect of the study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system; instead, they must accept it as a necessity. The level of acceptance by the users solely depends on the methods employed to educate them about the system and to make them familiar with it. Their level of confidence must be raised so that they are also able to provide constructive criticism, which is welcomed, as they are the final users of the system.

- **Economical feasibility**

This study is carried out to check the economic impact that the system will have on the organization. The amount of funds that the company can allocate to the research and development of the system is limited. The expenditures must be justified. Thus, the developed system is well within the budget, and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

7.2 SOFTWARE TESTING

The purpose of testing is to discover errors. Testing is the process of trying to uncover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies, and/or a finished product. It is the process of exercising software with the intent of ensuring that the software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of tests. Each test type addresses a specific testing requirement.

Types of Testing

There are many types of testing but the common and most efficient types of testing are:

1. Unit Testing
2. Integration Testing
3. System Testing
4. Acceptance Testing

- **Unit Testing**

Unit testing is testing the smallest testable unit of an application. It is done during the coding phase by the developers. Unit Testing helps in finding bugs early in the development process. Unit testing is the testing of an individual unit or group of related units. It falls under the class of white box testing. It is often done by the programmer to test that the unit he/she has implemented is producing expected output against given input.

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software

units of the application. It is done after the completion of an individual unit before integration.

Test Case ID	Test Description	Input	Expected Output	Status
TC01	Check whether the camera captures driver video	Start system	Camera opens and video stream starts	Pass
TC02	Verify system detects driver face in frame	Frame containing driver face	Face detected successfully	Pass
TC03	Verify eye region detection from face	Face image	Eye regions identified correctly	Pass
TC04	Verify model predicts driver state	Processed eye/face image	Output shows **Alert** or **Drowsy**	Pass
TC05	Verify alarm triggers when driver is drowsy	Model output = Drowsy	Warning alarm activated	Pass

Table 7.2.1: Unit testing

- **Integration Testing**

In Integration Testing different units, modules or components of a software application are tested as a combined entity. Integration tests are designed to test integrated software components to determine if they run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components. In the proposed system, the CNN-based drowsiness detection model is integrated with the user interface developed using Streamlit. Integration testing ensures that data flows smoothly from the user input in the front end to the prediction engine in the back end and that the results are accurately displayed. It validates the interaction between the ML model and the UI components, checking that the model receives the correct input format and returns predictions without errors. This testing also ensures that the user interface responds properly to various prediction outcomes, such as displaying the driver status as Alert or Drowsy.

Test Case ID	Test Description	Input	Expected Output	Status
IT01	Verify captured video frames are sent to processing module	Start camera and capture video	Frames successfully processed	Pass
IT02	Verify processed frames correctly detect driver face	Frame with driver face	Face detected accurately	Pass
IT03	Verify facial landmarks and eye features are extracted	Detected face image	Eye and facial features extracted	Pass
IT04	Verify extracted features are passed to CNN model for prediction	Eye/face features	Model predicts **Alert** or **Drowsy**	Pass
IT05	Verify alert system activates when drowsiness detected	Model output = Drowsy	Alarm or warning triggered	Pass

Table 7.2.2: Integration testing

- **System testing**

System Testing evaluates the overall functionality and performance of the complete software. It ensures all modules are integrated correctly and the application satisfies the end-user requirements**.** This testing phase simulates real-world usage scenarios to verify the system's behavior under expected and unexpected conditions. It includes functional testing, performance testing, security testing, and usability testing to assess the software from the user's perspective. In the context of the Driver Drowsiness Detection System real-time video capture, drowsiness detection, and alert generation. It also ensures that the application provides accurate outputs, handles invalid inputs gracefully, and performs efficiently across different devices or environments. System testing also checks the overall responsiveness and stability of the application under varying loads. It helps identify any bottlenecks or failures that may occur during prolonged or simultaneous usage.

Test Case ID	Test Scenario	Test Description	Input	Expected Output	Status
ST01	System Initialization	Verify the system starts correctly and initializes all modules	Start application	Camera and processing modules start successfully	Pass
ST02	Real-Time Monitoring	Verify system continuously monitors the driver	Live driver video	Frames processed continuously	Pass
ST03	Drowsiness Detection	Verify system detects driver drowsiness	Driver closes eyes for longer duration	System detects drowsy state	Pass
ST04	Alert Generation	Verify alarm activates when driver is drowsy	Model output = Drowsy	Alarm sound or warning displayed	Pass
ST05	System Shutdown	Verify system stops properly when user exits	Stop command	Application closes and camera stops	Pass

Table 7.2.3: System testing

- **Acceptance Testing**

Acceptance Testing is a method of software testing where a system is tested for acceptability. The main aim of this testing is to evaluate whether the system meets user requirements and is ready for deployment. It focuses on the external behavior of the system rather than internal logic. In this project, acceptance testing is performed by allowing users (drivers or end users) to interact with the system in real-time. The system is tested with different inputs such as live video feed, eye closure, and yawning conditions to verify accurate drowsiness detection and alert generation. In this project, acceptance testing is performed by allowing users (drivers or end users) to interact with the system in real-time. The system is tested with different inputs such as live video feed, eye closure, and yawning conditions to verify accurate drowsiness detection and alert generation.If any issues or missing features are identified, they can be addressed before final deployment. This testing marks the final step before the software is released for real-world use.

Test Case ID	Test Scenario	Test Description	Input	Expected Output	Status
AT01	System Startup	Verify system starts and initializes all modules correctly	Launch application	System loads successfully and camera starts	Pass
AT02	Driver Monitoring	Verify system monitors driver face in real time	Driver in front of camera	Driver face detected and monitored	Pass
AT03	Drowsiness Detection	Verify system detects driver drowsiness	Driver closes eyes or yawns	System identifies drowsy state	Pass
AT04	Alert Notification	Verify system alerts driver when drowsiness detected	Model output = Drowsy	Alarm sound or warning message triggered	Pass
AT05	User Satisfaction	Verify system works correctly and meets user requirements	Normal driving scenario	System monitors continuously and alerts when required	Pass

Table 7.2.4: Acceptance testing

Chapter -8

SOURCE CODE

Appendix A: Main Drowsiness Detection Code

File Name: drowsiness_detection.py

```
# Import the necessary packages
import datetime as dt
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from EAR_calculator import *
from imutils import face_utils
from imutils.video import VideoStream
import matplotlib.pyplot as plt
import matplotlib.animation as animate
from matplotlib import style
import imutils
import dlib
import time
import argparse
import cv2
from playsound import playsound
from scipy.spatial import distance as dist
import os
import csv
import numpy as np
import pandas as pd
from datetime import datetime
```

```
# ----------------------------
#Load CNN model
# ----------------------------

from tensorflow.keras.models import load_model

print("[INFO] Loading CNN model...")
cnn_model = load_model("eye_cnn_model.keras")
style.use('fivethirtyeight')
# Creating the dataset
def assure_path_exists(path):
    dir = os.path.dirname(path)
    if not os.path.exists(dir):
        os.makedirs(dir)

#all eye  and mouth aspect ratio with time
ear_list=[]
total_ear=[]
mar_list=[]
total_mar=[]
ts=[]
total_ts=[]
# Construct the argument parser and parse the arguments
ap = argparse.ArgumentParser()
ap.add_argument("-p", "--shape_predictor", required = True, help = "path to dlib's facial
landmark predictor")
ap.add_argument("-r", "--picamera", type = int, default = -1, help = "whether raspberry pi
camera shall be used or not")
args = vars(ap.parse_args())
```

```
# Declare a constant which will work as the threshold for EAR value, below which it will be regared as a blink
EAR_THRESHOLD = 0.3
# Declare another costant to hold the consecutive number of frames to consider for a blink
CONSECUTIVE_FRAMES = 20
# Another constant which will work as a threshold for MAR value
MAR_THRESHOLD = 14

# Initialize two counters
BLINK_COUNT = 0
FRAME_COUNT = 0

# Now, intialize the dlib's face detector model as 'detector' and the landmark predictor model as 'predictor'
print("[INFO]Loading the predictor.....")
detector = dlib.get_frontal_face_detector()
predictor = dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")

# Grab the indexes of the facial landamarks for the left and right eye respectively
(lstart, lend) = face_utils.FACIAL_LANDMARKS_IDXS["left_eye"]
(rstart, rend) = face_utils.FACIAL_LANDMARKS_IDXS["right_eye"]
(mstart, mend) = face_utils.FACIAL_LANDMARKS_IDXS["mouth"]

# Now start the video stream and allow the camera to warm-up
print("[INFO]Loading Camera.....")
vs = VideoStream(usePiCamera = args["picamera"] > 0).start()
time.sleep(2)

assure_path_exists("dataset/")
count_sleep = 0
```

```
count_yawn = 0


# Now, loop over all the frames and detect the faces
while True:
    # Extract a frame
    frame = vs.read()
    cv2.putText(frame, "PRESS 'q' TO EXIT", (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 3)
    # Resize the frame
    frame = imutils.resize(frame, width = 500)
    # Convert the frame to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    # Detect faces
    rects = detector(frame, 1)

    # Now loop over all the face detections and apply the predictor
    for (i, rect) in enumerate(rects):
        shape = predictor(gray, rect)
        # Convert it to a (68, 2) size numpy array
        shape = face_utils.shape_to_np(shape)

        # Draw a rectangle over the detected face
        (x, y, w, h) = face_utils.rect_to_bb(rect)
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
        # Put a number
        cv2.putText(frame, "Driver", (x - 10, y - 10), cv2.FONT_HERSHEY_SIMPLEX,
0.5, (0, 255, 0), 2)

        leftEye = shape[lstart:lend]
        rightEye = shape[rstart:rend]
```

```
mouth = shape[mstart:mend]
# Compute the EAR for both the eyes
leftEAR = eye_aspect_ratio(leftEye)
rightEAR = eye_aspect_ratio(rightEye)

# Take the average of both the EAR
EAR = (leftEAR + rightEAR) / 2.0
#live datawrite in csv
ear_list.append(EAR)
#print(ear_list)
ts.append(dt.datetime.now().strftime('%H:%M:%S.%f'))
# Compute the convex hull for both the eyes and then visualize it
leftEyeHull = cv2.convexHull(leftEye)
rightEyeHull = cv2.convexHull(rightEye)
# Draw the contours
cv2.drawContours(frame, [leftEyeHull], -1, (0, 255, 0), 1)
cv2.drawContours(frame, [rightEyeHull], -1, (0, 255, 0), 1)
cv2.drawContours(frame, [mouth], -1, (0, 255, 0), 1)

MAR = mouth_aspect_ratio(mouth)
mar_list.append(MAR/10)
# Check if EAR < EAR_THRESHOLD, if so then it indicates that a blink is taking
place
# Thus, count the number of frames for which the eye remains closed
if EAR < EAR_THRESHOLD:
    FRAME_COUNT += 1

    cv2.drawContours(frame, [leftEyeHull], -1, (0, 0, 255), 1)
    cv2.drawContours(frame, [rightEyeHull], -1, (0, 0, 255), 1)

    if FRAME_COUNT >= CONSECUTIVE_FRAMES:
```

```
            count_sleep += 1
            # Add the frame to the dataset ar a proof of drowsy driving
            cv2.imwrite("dataset/frame_sleep%d.jpg" % count_sleep, frame)
            playsound('sound files/alarm.mp3')
            cv2.putText(frame, "DROWSINESS ALERT!", (270, 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
        else:
            if FRAME_COUNT >= CONSECUTIVE_FRAMES:
                playsound('sound files/warning.mp3')
            FRAME_COUNT = 0
        #cv2.putText(frame, "EAR: {:.2f}".format(EAR), (300, 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

        # Check if the person is yawning
        if MAR > MAR_THRESHOLD:
            count_yawn += 1
            cv2.drawContours(frame, [mouth], -1, (0, 0, 255), 1)
            cv2.putText(frame, "DROWSINESS ALERT!", (270, 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
            # Add the frame to the dataset ar a proof of drowsy driving
            cv2.imwrite("dataset/frame_yawn%d.jpg" % count_yawn, frame)
            playsound('sound files/alarm.mp3')
            playsound('sound files/warning_yawn.mp3')
    #total data collection for plotting
    for i in ear_list:
        total_ear.append(i)
    for i in mar_list:
        total_mar.append(i)
    for i in ts:
        total_ts.append(i)
    #display the frame
```

```
    cv2.namedWindow("Output", cv2.WINDOW_NORMAL)  # allow resizing
    cv2.setWindowProperty("Output", cv2.WND_PROP_FULLSCREEN,
cv2.WINDOW_FULLSCREEN)  # full screen
    cv2.imshow("Output", frame)

    key = cv2.waitKey(1) & 0xFF




    if key == ord('q'):
        break

import matplotlib.pyplot as plt

# Epochs
epochs = list(range(1, 16))

# Training values from your logs
train_accuracy = [0.8693, 0.9622, 0.9730, 0.9774, 0.9874, 0.9884, 0.9901, 0.9924,
0.9931, 0.9932, 0.9956, 0.9966, 0.9931, 0.9937, 0.9976]
val_accuracy   = [0.5432, 0.5342, 0.5878, 0.9286, 0.9598, 0.8348, 0.9643, 0.9241,
0.9568, 0.9955, 0.9167, 0.7128, 0.9643, 0.9836, 0.9747]

train_loss = [0.4089, 0.1063, 0.0703, 0.0657, 0.0403, 0.0366, 0.0274, 0.0277, 0.0253,
0.0199, 0.0150, 0.0156, 0.0241, 0.0191, 0.0079]
val_loss   = [0.6230, 1.9475, 1.8781, 0.1412, 0.0975, 0.4692, 0.0961, 0.2222, 0.1128,
0.0119, 0.2515, 1.0128, 0.1343, 0.0578, 0.0537]

# Accuracy plot
plt.figure(figsize=(12,5))
plt.plot(epochs, train_accuracy, 'o-', label='Training Accuracy')
```

```
plt.plot(epochs, val_accuracy, 's-', label='Validation Accuracy')
plt.title('Training vs Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.grid(True)
plt.legend()
plt.show()

# Loss plot
plt.figure(figsize=(12,5))
plt.plot(epochs, train_loss, 'o-', label='Training Loss')
plt.plot(epochs, val_loss, 's-', label='Validation Loss')
plt.title('Training vs Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)
plt.legend()
plt.show()

cv2.destroyAllWindows()
vs.stop()
```

Appendix B: Eye Aspect Ratio Calculation

File Name: EAR_calculator.py

```
# Define the function for calculating the Eye Aspect Ratio(EAR)
from scipy.spatial import distance as dist
def eye_aspect_ratio(eye):
    # Vertical eye landmarks
    A = dist.euclidean(eye[1], eye[5])
    B = dist.euclidean(eye[2], eye[4])
```

```
    # Horizontal eye landmarks
    C = dist.euclidean(eye[0], eye[3])

    # The EAR Equation
    EAR = (A + B) / (2.0 * C)
    return EAR

def mouth_aspect_ratio(mouth):
    A = dist.euclidean(mouth[13], mouth[19])
    B = dist.euclidean(mouth[14], mouth[18])
    C = dist.euclidean(mouth[15], mouth[17])

    MAR = (A + B + C) / 3.0
    return MAR
```

Appendix C: CNN Model Training

File Name: train.py

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout,
BatchNormalization
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

# ----------------------------
# CONFIG
# ----------------------------
IMG_SIZE = 64
BATCH_SIZE = 32
```

```
# ----------------------------
# DATA AUGMENTATION
# ----------------------------
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True,
    fill_mode='nearest'
)

val_datagen = ImageDataGenerator(rescale=1./255)

train_data = train_datagen.flow_from_directory(
    "dataset/train",
    target_size=(IMG_SIZE, IMG_SIZE),
    color_mode="grayscale",
    class_mode="binary",
    batch_size=BATCH_SIZE
)

val_data = val_datagen.flow_from_directory(
    "dataset/val",
    target_size=(IMG_SIZE, IMG_SIZE),
    color_mode="grayscale",
    class_mode="binary",
    batch_size=BATCH_SIZE
)
```

```
# ----------------------------
# MODEL
# ----------------------------
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(IMG_SIZE, IMG_SIZE, 1)),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),  # reduced from 128 to 64
    BatchNormalization(),
    MaxPooling2D(2,2),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

```
# ----------------------------
# CALLBACKS
# ----------------------------
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, verbose=1)

# ----------------------------
# TRAINING
# ----------------------------
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=50,
    callbacks=[early_stop, reduce_lr]
)

# ----------------------------
# SAVE MODEL
# ----------------------------
model.save("eye_cnn_model.keras")
```

Appendix D: Graph Generation

File Name: graphs.py

```
import matplotlib.pyplot as plt

# Epochs
epochs = list(range(1, 16))

# Training values from your logs
```

```
train_accuracy = [0.8693, 0.9622, 0.9730, 0.9774, 0.9874, 0.9884, 0.9901, 0.9924,
0.9931, 0.9932, 0.9956, 0.9966, 0.9931, 0.9937, 0.9976]
val_accuracy   = [0.5432, 0.5342, 0.5878, 0.9286, 0.9598, 0.8348, 0.9643, 0.9241,
0.9568, 0.9955, 0.9167, 0.7128, 0.9643, 0.9836, 0.9747]

train_loss = [0.4089, 0.1063, 0.0703, 0.0657, 0.0403, 0.0366, 0.0274, 0.0277, 0.0253,
0.0199, 0.0150, 0.0156, 0.0241, 0.0191, 0.0079]
val_loss   = [0.6230, 1.9475, 1.8781, 0.1412, 0.0975, 0.4692, 0.0961, 0.2222, 0.1128,
0.0119, 0.2515, 1.0128, 0.1343, 0.0578, 0.0537]

# Accuracy plot
plt.figure(figsize=(12,5))
plt.plot(epochs, train_accuracy, 'o-', label='Training Accuracy')
plt.plot(epochs, val_accuracy, 's-', label='Validation Accuracy')
plt.title('Training vs Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.grid(True)
plt.legend()
plt.show()

# Loss plot
plt.figure(figsize=(12,5))
plt.plot(epochs, train_loss, 'o-', label='Training Loss')
plt.plot(epochs, val_loss, 's-', label='Validation Loss')
plt.title('Training vs Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)
plt.legend()
plt.show()
```

Appendix E: GUI Interface (Tkinter)

File Name: index.py

```
from tkinter import *
from flask import Flask,redirect, url_for,render_template,request
import os

def d_dtcn():
    root = Tk()
    root.configure(background = "white")

    def function1():
        os.system("python drowsiness_detection.py --shape_predictor
shape_predictor_68_face_landmarks.dat")
        exit()
    root.title("DROWSINESS DETECTION")
    Label(root, text="DROWSINESS DETECTION",font=("times new
roman",20),fg="black",bg="aqua",height=2).grid(row=2,rowspan=2,columnspan=5,stick
y=N+E+W+S,padx=5,pady=10)
    Button(root,text="Run using web cam",font=("times new
roman",20),bg="#0D47A1",fg='white',command=function1).grid(row=5,columnspan=5,s
ticky=W+E+N+S,padx=5,pady=5)

    Button(root,text="Exit",font=("times new
roman",20),bg="#0D47A1",fg='white',command=root.destroy).grid(row=9,columnspan=
5,sticky=W+E+N+S,padx=5,pady=5)

    root.mainloop()
```

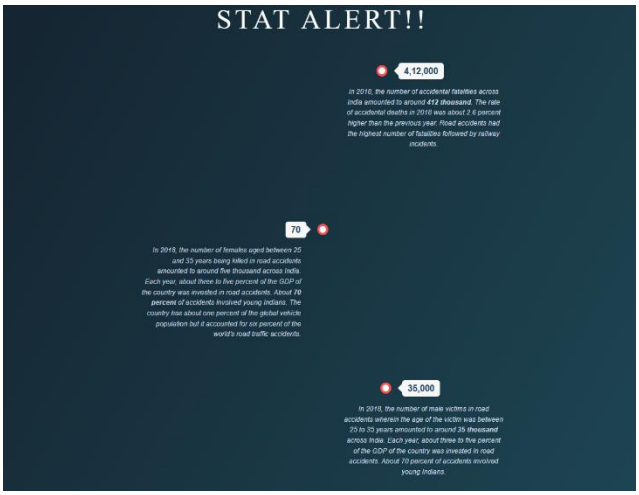
Chapter 9

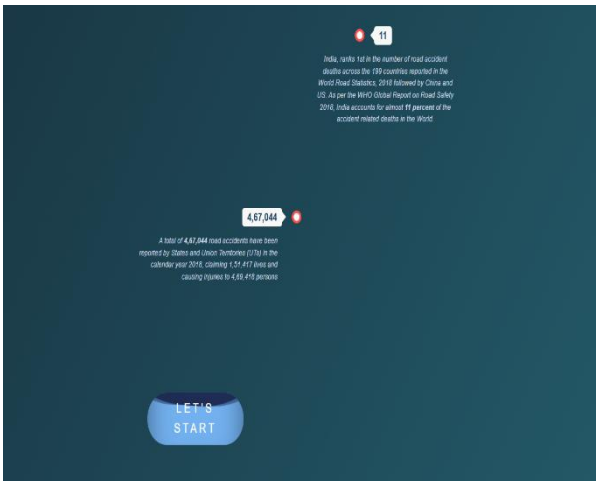
SCREEN LAYOUTS

9.1 INTERFACE LAYOUT:

9.11 Web Interface:

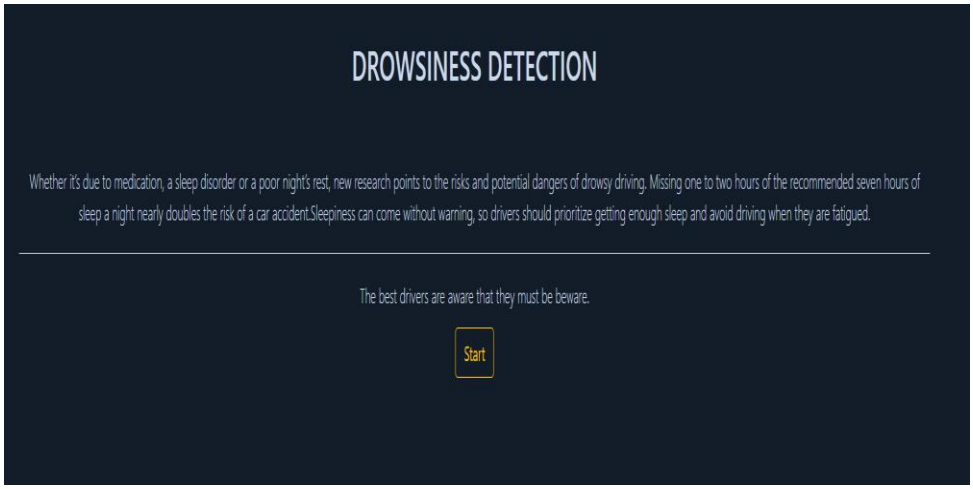
Web Interface of Driver Drowsiness Detection System



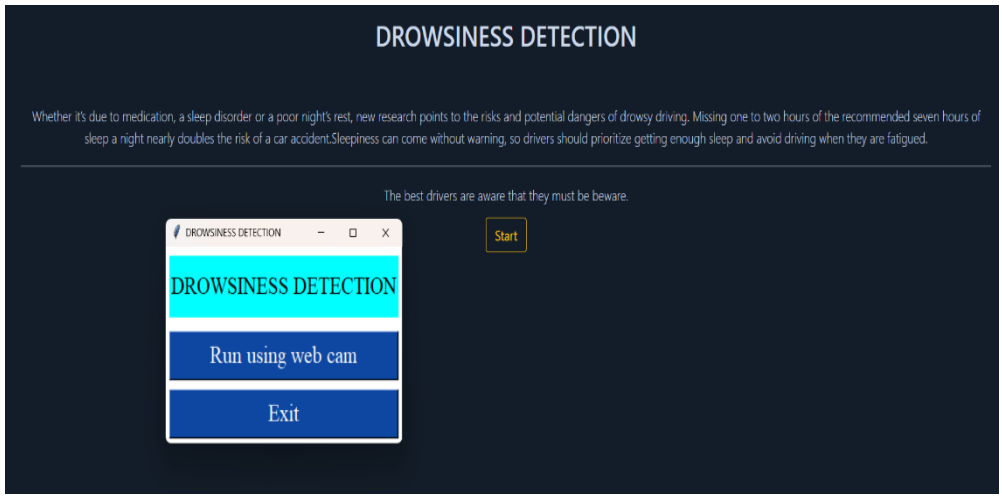


9.12 Detection GUI Window:

Detection GUI window is displayed after clicking the start button.

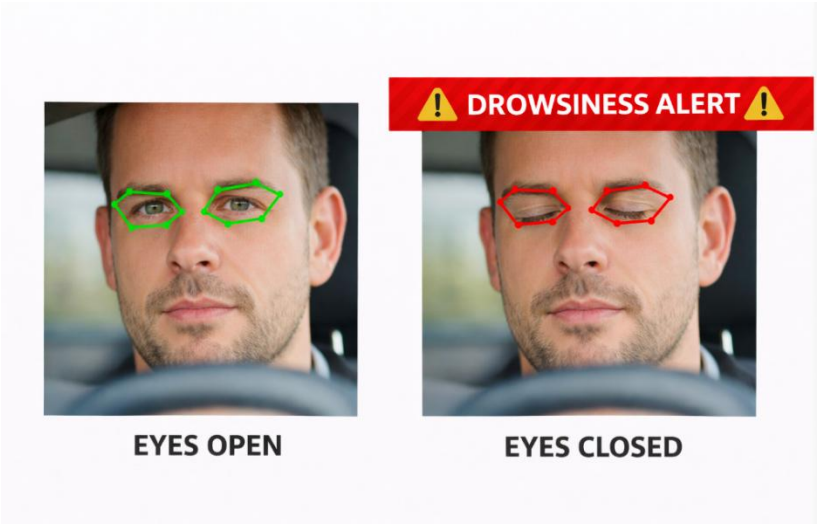


It is developed using Tkinter and provides options such as “Run using web cam” to start real-time drowsiness detection and “Exit” to close the application.



9.2 RESULT:

9.2.1 Eye State Detection (Eyes Open and Eyes Closed with Alert)



Output-1

9.2.2 Yawning Detection (Mouth Closed and Yawning with Alert)

Output-2

Chapter 10

FUTURE ENHANCEMENTS

The Real-Time Driver Drowsiness Detection System can be further improved by integrating advanced technologies and additional features to enhance accuracy, usability, and safety. Some of the possible future enhancements are as follows:

- **Integration with Smart Vehicles**

The system can be integrated directly into modern vehicles and smart transportation systems to continuously monitor driver behavior and automatically take preventive actions.

- **Mobile Application Integration**

A mobile application can be developed to monitor driver alertness, store driving data, and send notifications or alerts to the driver's smartphone.

- **Advanced Deep Learning Models**

More advanced deep learning architectures such as ResNet, EfficientNet, or Vision Transformers can be used to improve the accuracy and robustness of drowsiness detection.

- **Multiple Driver Monitoring**

The system can be enhanced to support multiple cameras to monitor the driver from different angles, improving detection accuracy in different driving conditions.

- **Automatic Vehicle Control**

In critical situations, the system can be connected with the vehicle control system to reduce vehicle speed or safely stop the vehicle if the driver does not respond to alerts.

- **Cloud-Based Data Storage and Analysis**

Driver alertness data can be stored in the cloud to analyze driving patterns, generate reports, and improve long-term safety monitoring.

- **Integration with GPS and Emergency Alerts**

The system can be integrated with GPS tracking to send emergency alerts or location information to family members or authorities if the driver becomes unresponsive.

These enhancements will make the system more intelligent, reliable, and suitable for real-world applications in modern vehicles and transportation systems.

Chapter 11

CONCLUSION

The Real-Time Driver Drowsiness Detection and Alert System is developed to improve road safety by monitoring the driver's alertness and detecting signs of fatigue during driving. Driver drowsiness is one of the major causes of road accidents, and early detection of fatigue can help prevent dangerous situations. The proposed system uses computer vision and deep learning techniques to continuously analyze the driver's facial features such as eye closure, blinking patterns, and yawning.

The system uses a camera to capture real-time video of the driver and processes each frame using image preprocessing and feature extraction techniques. A trained Convolutional Neural Network (CNN) model is used to classify the driver's state as either alert or drowsy. When the system detects signs of drowsiness, it immediately activates an alert mechanism, such as an audio alarm, to warn the driver and help them regain attention.

The results show that the system can effectively detect driver fatigue in real time and provide timely alerts to reduce the risk of accidents. The proposed solution is cost-effective, non-intrusive, and easy to implement, making it suitable for integration into modern vehicles and intelligent transportation systems.

Overall, this project demonstrates how artificial intelligence and deep learning technologies can be used to develop a reliable driver monitoring system that enhances road safety and helps prevent accidents caused by driver fatigue.

CHAPTER 12

BIBILOGRAPHY

1. Hassan, O. F., Ibrahim, A. F., Gomaa, A., & Makhlouf, M. A., "Real-Time Driver Drowsiness Detection Using Transformer Architectures: A Novel Deep Learning Approach," Scientific Reports, vol. 15, 2025.
2. Fonseca, T., & Ferreira, S., "Drowsiness Detection in Drivers: A Systematic Review of Deep Learning-Based Models," Applied Sciences, vol. 15, no. 16, 2025.
3. Harb, H., "An Efficient Drowsiness Detection Framework for Improving Driver Safety Through Supervised Learning Models," World Electric Vehicle Journal, vol. 16, no. 11, 2025.
4. Madane, A., Singh, A., Fargade, S., & Dongare, A., "Real-Time Driver Drowsiness Detection Using Deep Learning and Computer Vision Techniques," International Journal of Scientific Research in Science, Engineering and Technology, vol. 12, no. 3, 2025.
5. Bhatt, Y. V., & Joshi, N. A., "State-of-the-Art in Driver's Drowsiness Detection: A Comprehensive Survey," Journal of Information Systems Engineering and Management, 2025.
6. Beles, H., Vesselenyi, T., Rus, A., Mitran, T., Scurt, F. B., & Tolea, B. A., "Driver Drowsiness Multi-Method Detection for Vehicles with Autonomous Driving Functions," Sensors, vol. 24, no. 5, 2024.
7. Díaz-Santos, S., Cigala-Álvarez, Ó., Gonzalez-Sosa, E., Caballero-Gil, P., & Caballero-Gil, C., "Driver Identification and Detection of Drowsiness while Driving," Applied Sciences, vol. 14, no. 6, 2024.
8. Siddhad, G., Dey, S., & Roy, P. P., "DrowzEE-G-Mamba: Leveraging EEG and State Space Models for Driver Drowsiness Detection," arXiv preprint, 2024.